

Avatar System

Build Blueprint & Recovery Doc · v2 · complete process + working code

What this is

Avatars are supernatural / metaphysical ethereal spirits — the user's living embodiment and storyteller. GOAL: a procedural tool that mixes and matches separate Bodies, Heads, and Eyes to generate random avatars from a seed (the El-Fish gene model, minus breeding). This document is the single source of truth: spec, drawings, method, traps, and complete code.

Aesthetic Law (hard rule)

No solid objects. Every part is simple math worn as an ethereal effect. A primitive is only the invisible *region* a material fills. **Math defines WHERE; effect defines WHAT IT LOOKS LIKE** — always decoupled. Nothing is a modelled mesh asset: every shape is a formula.

Framework & scope

6 Forms (Form 6 is a candidate that may replace Form 1) · **4 Parts**: Eye, Head, Body, Sparks. Head and Eye share the same 5 shapes → **5 × 5 = 25 head/eye combinations**. Traits / Behaviors / Expressions still to define.

Universal constants

- A bright **star core = the eye**, sitting in the head.
- **Veil / wisp / smoke** material, never a surface.
- Base-point glow (ground / water) is an **effect, not part of the avatar**.
- Bilateral or radial symmetry. Every form has a head and a tail.

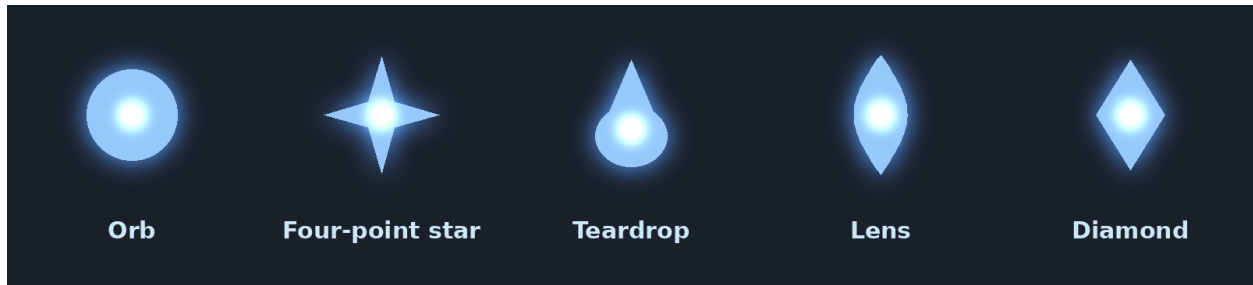
The Parts (LOCKED)

A part must be genuinely SEPARABLE (independently swappable) — that is what lets the tool mix and match. **Tail** and **Wisps** FAILED the test (both flow continuously out of the body; you cannot graft a foreign tail on seamlessly) and are folded into Body. **Aura / glow** is a trait, not a part.

1. **Eye** — the star-core spark. DEFINED.
2. **Head** — the vessel the eye sits in. DEFINED.
3. **Body** — the entire continuous flowing volume: silhouette, tail, wisps. DEFINED for Form 1 (see code).
4. **Sparks** — tiny free-floating points along the tendrils. DEFINED.

Part 1 — The Eye

The eye is the star core, sitting in the head. **Count fixed at 1. Shape varies across 5:**



Orb · Four-point star · Teardrop · Lens · Diamond

Star flare (light effect, not shape)

A bright core needs a **star flare** to read as a light source — this is what the eye was missing. Baked into the sprite texture: 4 long tapered spikes in a cross + 4 shorter dimmer diagonals, then a hard bright core composited over with `globalCompositeOperation='lighter'`. Because it is a sprite it always faces camera. Treat flare as a TRAIT any eye shape can wear.

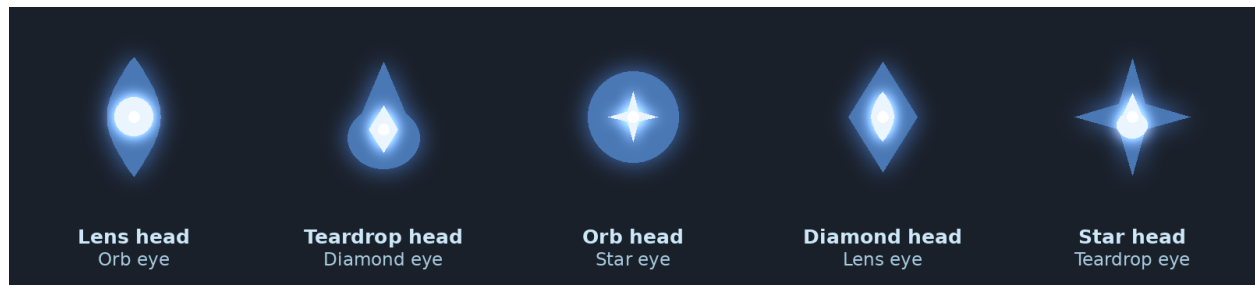
Eye movement — the saccade (ported from Lonnie's world build)

It picks a spot, moves to it, then holds dead still for an **uneven** beat before glancing elsewhere; occasionally it startles and looks much further. The randomised hold is what stops it reading as a scanning machine.

```
angle = random × 2π  
dist = √random × reach × EYE_LOOK (reach = 1.0 if startle, else 0.55)  
target = (cos·dist, sin·dist)  
pos += (target - pos) × (1 - e-(DART_SPEED × delta))  
√random spreads glances evenly over the area instead of clustering them at the centre.  
Avatar 1 settings: EYE_LOOK 0.13 · DART_SPEED 5.5 (calm glide; 22 read as scared) · HOLD  
1.2–5.5s · STARTLE 6%
```

Part 2 — The Head

The head is a REAL separate object — the **vessel the eye sits in** — so the tool can mix and match. Same 5 shapes as the eye; any head pairs with any eye.



The head as MATH (no mesh, no modelling)

$R(a) = R_i + (R_o - R_i) \cdot (0.5 + 0.5 \cdot \cos(P \cdot a))^K$ ← smooth star outline, no straight edges
 $th(u) = T \cdot e^{-(u/W)^N} \cdot (1 - u^2)$ ← ONE continuous profile: flat face, organic rolloff
 $z = curve \cdot (x^2 + y^2)$ ← curls back to follow the body

Why each term matters. K controls point sharpness — low K reads as a *plus sign*, high K gives true star points with a round centre. R_i sets how much open centre the eye has to move in. The thickness must be a **single analytic curve**: a flat plateau spliced to a taper leaves a visible crease ring, and a simple $(1-u)^q$ peaks at the centre so the four points converge to a pinch — no flat face for the eye. The super-gaussian is naturally flat in the middle and rolls off smoothly with no join anywhere.

Avatar 1 head: R_o 1.0 · R_i 0.46 · P 4 · K 3.2 · T 0.20 · W 0.78 · N 5.0 · size 0.56 · curve 0.40 · position z -0.02 (slightly intersecting the body)

Head material

Opaque, flat colour, emissive fresnel glow. Colour matches the body mint (0x7fecd4). Glow comes from the head's OWN shader — no post-process, no sprite:

```
fres = pow(1 - |dot(normal, viewDir)|, 2.2)
col = uColor × (0.85 + 1.60·fres)
col = mix(col, white, fres × 0.45) ← hottest rim bleaches toward white
```

The surface must be OPAQUE. See the traps below — this was the single most expensive lesson of the session.

Part 4 — Sparks

The one genuinely separable part: discrete free points that attach to ANY form as an independent particle layer without touching the body.

- **Spawn ALONG the tendrils** — collect sample points from every strand curve into SPARK_ANCHORS during build, then place each spark at a random anchor + jitter. (Clustering them in a ball reads wrong.)
- THREE.Points + ShaderMaterial, additive.
- Tiny SHARP points; **hard-clamp point size** so additive cannot blow out.
- Barely drift; mostly hold position and twinkle.
- Warm gold / white + a few teal.

```
Count ~220 · size 1.4-3.4 · jitter 0.06 · drift ±0.02 · twinkle 0.5+0.5·sin(t·3.2)
gl_PointSize = clamp(size·(0.8+0.6·tw)·(8/-mv.z)·dpr, 1.5, 7·dpr)
alpha = pow(core, 2.2) · (0.8 + 0.5·tw)
```

TRAPS — every mistake this session, so they are never repeated

Rendering (the expensive ones)

- **Translucent + depthWrite = GAPS.** A see-through surface that writes depth draws its own triangles in arbitrary order; whichever lands first claims the depth and slices of the surface vanish. This produced the see-through wedges in the head.
- **Translucent + no depthWrite = LAYERING.** You see the far side of the shell through the near side. Adding a halo pass made it worse (4 stacked surfaces).
- **Hashed / dithered alpha** to soften the edge: reintroduces the same gap artifact. Rejected.
- **Depth pre-pass** works technically but was rejected as not viable for the real world build.
- **CONCLUSION: the head must be OPAQUE.** Get glow from its own emissive shader.
- **Additive stacks to white.** Never additive for the body; only tiny, sparse, size-clamped points (sparks) and the eye core.
- **Sprite canvas filled with opaque black** → a black rectangle over the scene (it writes alpha into the framebuffer). Leave sprite canvases transparent.
- **Eye sprite with depthTest ON** pops in and out — a billboard is one flat depth, so it cannot be partially occluded. Left OFF.
- **Body strands drawn over the head** made it look invisible from the front — it was there all along.

Shape / form

- A radial brightness gradient across a surface reads as **lit shading on a solid object** — wrong for these creatures. Use flat colour + soft edges.
- Stacked offset planes read as stacked planes, not an extruded object.
- An extruded, bevelled, lit mesh reads as a **blocky primitive** — wrong for an organic spirit. Describe the shape as math instead.
- Piecewise thickness (flat plateau + taper) leaves a **crease ring**. Use one analytic curve.
- Low star exponent K reads as a **plus sign**.
- Thin strands tapering to points read as **spider legs**. Use thick locks with soft wide ends.
- Inventing shape that is not in the drawing (a spiky head that was never there).

Process

- **Use a debug colour (red) early** when a part is not visible — it separates 'not rendering' from 'not visible against the body'.
- **Save a RESTORE POINT file before risky changes** so a bad idea is one command to undo.
- **Ask before changing things** that were already approved.
- Test files are tests: do not treat them as saved assets until they are approved.

AVATAR 1 — the completed worked example

The first fully assembled avatar, and the template for every one that follows. File: `wisp_avatar1.html` (full source in Appendix A).

Body

Flowing tendrils. Each strand: fan = $F \cdot \sqrt{t}$ (spread from the core) · sweep = $S \cdot t$ (flow back along the tail axis) · droop = $D \cdot t^2$ (gravity) · plus multi-harmonic sine sway. Rendered as **cross-ribbons** (two perpendicular planes per strand) so they hold volume from any camera angle. All strands merged into ONE BufferGeometry and waved in the vertex shader. Normal blending. Three populations: mane (58, short, electric blue on top), hair (58, long teal locks), bold (13, hero tendrils).

Head

Math star (above): 4-point, K 3.2, flat face for the eye, curved back, opaque, emissive fresnel glow, body mint.

Eye

Orb core sprite with star flare, additive, depthTest off. Scale 0.52 (+0.06 pulse), seated at z 0.06 in the head's flat face. Saccade look-around, calm settings.

Sparks

~220 points spawned along the tendrils, tiny, sharp, twinkling.

Build order that worked

1. Body first (it defines the silhouette and gives the head somewhere to sit).
2. Head as math, in DEBUG RED until the shape is right.
3. Eye seated in the head's flat face; add flare; then tune movement.
4. Sparks last, anchored to the finished tendrils.
5. Merge into one file and tune the whole creature together.

The Forms

Front and side lock each form's true 3D silhouette.

Form 1 · Comet / Wisp



FRONT



SIDE

Spec: Head = central core. Tail = flowing hair swept to one side. BUILT — see Avatar 1 and Appendix A.

Status: Front + side locked · BUILT

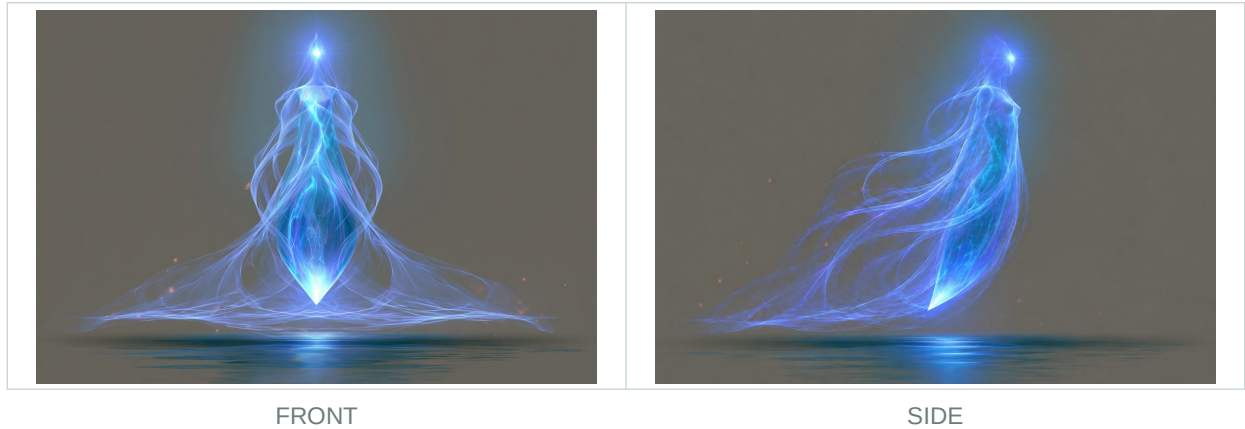
Form 2 · Rising Plume



Spec: Head = central core. Tail streams off to one side. Bright point at base = GROUND EFFECT, not the avatar.

Status: Front + side locked

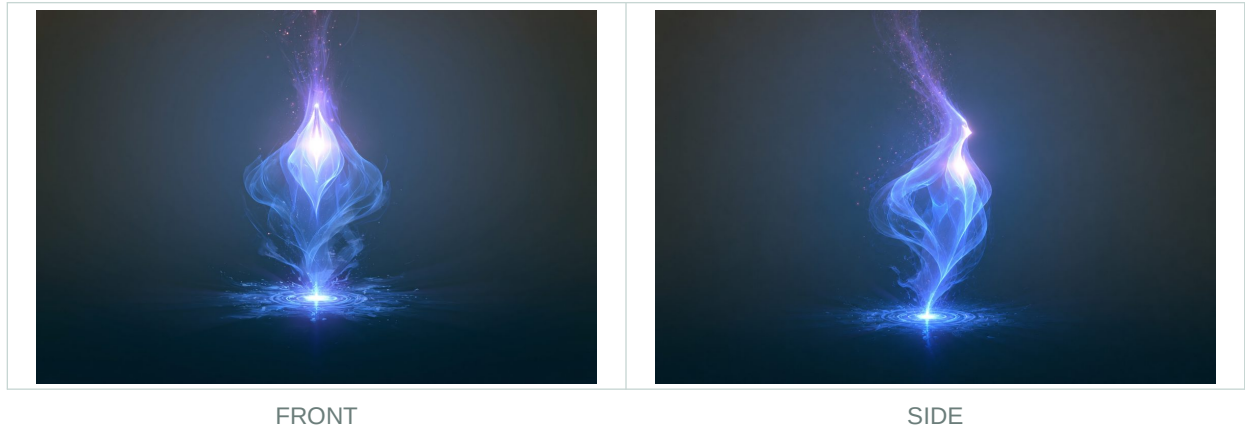
Form 3 · Veiled Figure



Spec: Crown-star above the head; central body-core; veils drape back and down as the tail, tapering to a point. Water glow = reflection effect.

Status: Front + side locked

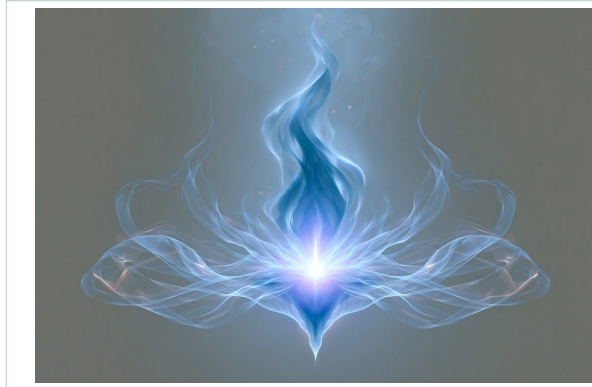
Form 4 · Flame-bud / Teardrop



Spec: Head = bright core in the upper bud. Tail = smoke/sparks streaming upward. Water ripple = effect.

Status: Front + side locked

Form 5 · Lotus Bloom



FRONT



SIDE

Spec: Head = bright core at the leading tip. Tail = veils streaming back. Radial bloom head-on; a creature gliding side-on.

Status: Front + side locked

Form 6 · Filament Wing (Gossamer Bird)



Spec: Fine-filament winged spirit. Front = symmetric wing-burst, gold core; side = gliding profile, head-core leading.

Status: CANDIDATE — may replace Form 1

How we build (recovery method)

Drawing → working generator

1. Name the archetype and its job.
2. Get front + side references — two silhouettes define the 3D reading.
3. Find the unifying 3D form (two 2D drawings are one 3D object from two angles).
4. Describe the form as MATH, not assumptions.
5. Make it real 3D early (cross-ribbons, not billboards).
6. Animate on the GPU (one merged geometry, vertex-shader displacement).
7. Tune to the drawing, not a generic idea.
8. Randomise a seed LAST, once one gene set holds up.

Working principles

- Fix problems by removing the cause, not by adding patches.
- Define and plan before building — a tech demo is not a system.
- Lead with the researched, working path; flag limitations up front.
- Diagrams over text.
- Write work-in-progress to files immediately; never let decisions live only in a chat.
- Be scrupulously accurate about what is and is not saved.

Progress & what's next

DONE: 6 Forms (front + side) · Eye (5 shapes + flare + saccade) · Head (5 shapes, math, mix & match) · Sparks · **Avatar 1 fully assembled**. NEXT: decide Form 6 vs Form 1 · define the 5 Traits, 5 Behaviors, 5 Expressions · seed / gene encoding · a body generator for Forms 2–6 · then the procedural tool itself.

Appendix A — AVATAR 1 complete source: wisp_avatar1.html

Read verbatim from the working file. Body + head + eye + sparks, self-contained, three.js r128.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
<title>Whisp Creature 3D</title>
<style>
* { margin:0; padding:0; box-sizing:border-box; }
html, body { width:100%; height:100%; overflow:hidden; }
body { background: radial-gradient(circle at 50% 48%,
#566d68 0%, #445656 36%, #333f3f 78%, #262f2f 100%);
cursor: grab; font-family: system-ui, sans-serif; }
body.drag { cursor: grabbing; }
#app { position:fixed; inset:0; }
#ui { position:fixed; top:14px; left:0; right:0; display:flex; gap:8px; justify-content:center; z-index:5; }
.btn { font:400 12px/1 system-ui,sans-serif; letter-spacing:.14em; text-transform:uppercase;
color:rgba(215,250,242,.6); background:rgba(20,40,40,.35);
border:1px solid rgba(150,240,220,.25); border-radius:20px;
padding:8px 18px; cursor:pointer; backdrop-filter:blur(4px); transition:all .2s ease; }
.btn:hover { color:rgba(230,255,248,.9); border-color:rgba(150,240,220,.5); }
.btn.on { color:#0a1a1a; background:rgba(150,240,220,.85); border-color:rgba(150,240,220,.9); }
#hint { position:fixed; bottom:22px; left:0; right:0; text-align:center;
font:300 12px/1 system-ui,sans-serif; letter-spacing:.14em; text-transform:uppercase;
color:rgba(210,250,240,.5); pointer-events:none; transition:opacity 1.4s ease; }
#hint.fade { opacity:0; }
</style>
</head>
<body>
<div id="app"></div>
<div id="ui">
<button class="btn on" id="bFront">Front</button>
<button class="btn" id="bSide">Side</button>
</div>
<div id="hint">drag to rotate &nbsp;&nbsp;  scroll to zoom</div>
<script src="https://cdn.jsdelivr.net/npm/three.js/r128/three.min.js"></script>
<script>
(function () {
const container = document.getElementById('app');
const renderer = new THREE.WebGLRenderer({ antialias:true, alpha:true });
renderer.setSize(innerWidth, innerHeight);
renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
renderer.setClearColor(0x000000, 0);
container.appendChild(renderer.domElement);

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(55, innerWidth/innerHeight, 0.1, 100);
const group = new THREE.Group();
scene.add(group);

// palette - reads as soft colored hair on a light page
const CORE = new THREE.Color(0x9fecdd); // bright mint-cyan core (not pure white)
const MINT = new THREE.Color(0x7fecdd);
const TEAL = new THREE.Color(0x2ba895);
const DEEP = new THREE.Color(0x1c8f7e);
const BLUE = new THREE.Color(0x9fd4ff);
const BLUEL = new THREE.Color(0xe0f0ff);

// ----- merged strand engine (real 3D cross-ribbons, GPU-animated) -----
```

```

const P=[], CR=[], LN=[], COL=[], WV=[], WP=[];
const SPARK_ANCHORS=[];
function norm(x,y,z){ const l=Math.hypot(x,y,z)||1; return [x/l,y/l,z/l]; }

function addStrand(cpts, opt){
  const M = opt.M || 18;
  const curve = new THREE.CatmullRomCurve3(cpts.map(p=>new THREE.Vector3(p[0],p[1],p[2])));
  const sp = curve.getSpacedPoints(M-1);
  if(SPARK_ANCHORS.length<8000){ for(let ai=Math.floor((M-1)*0.3); ai<Math.floor((M-1)*0.92); ai+=2){ SPARK_ANCHORS.push([sp[ai].x,sp[ai].y,sp[ai].z]); } }
  const bx=[],by=[],bz=[],s=[],N1=[],N2=[],w=[],col=[];
  for(let i=0;i<M;i++){ bx[i]=sp[i].x; by[i]=sp[i].y; bz[i]=sp[i].z; s[i]=i/(M-1); }
  for(let i=0;i<M;i++){
    const i0=i>0?i-1:0, i1=i<M-1?i+1:M-1;
    let T=norm(bx[i1]-bx[i0], by[i1]-by[i0], bz[i1]-bz[i0]);
    let ux=0,uy=1,uz=0;
    let n1x=T[1]*uz-T[2]*uy, n1y=T[2]*ux-T[0]*uz, n1z=T[0]*uy-T[1]*ux;
    if(Math.hypot(n1x,n1y,n1z)<0.02){ ux=1;uy=0;uz=0;
      n1x=T[1]*uz-T[2]*uy; n1y=T[2]*ux-T[0]*uz; n1z=T[0]*uy-T[1]*ux; }
    let n1=norm(n1x,n1y,n1z);
    let n2=norm(T[1]*n1[2]-T[2]*n1[1], T[2]*n1[0]-T[0]*n1[2], T[0]*n1[1]-T[1]*n1[0]);
    N1.push(n1); N2.push(n2);
    w.push(opt.width * (0.45 + 0.55*Math.pow(1-s[i], 1.2))); // stays thick, soft wide end (no needle point)
    // 3-stop color: bright core -> mid -> deep tip
    let c; const ss=s[i];
    if(ss<0.5) c=opt.cRoot.clone().lerp(opt.cMid, ss*2);
    else c=opt.cMid.clone().lerp(opt.cTip, (ss-0.5)*2);
    col.push(c);
  }
  const wd = N1[Math.floor(M/2)];
  const wp = [opt.speed, opt.waves, opt.amp, opt.phase];
  function plane(N){
    const C=[],L=[],R=[];
    for(let i=0;i<M;i++){
      C[i]=[bx[i],by[i],bz[i]];
      L[i]=[bx[i]+N[i][0]*w[i], by[i]+N[i][1]*w[i], bz[i]+N[i][2]*w[i]];
      R[i]=[bx[i]-N[i][0]*w[i], by[i]-N[i][1]*w[i], bz[i]-N[i][2]*w[i]];
    }
    const pv=(Pt,cross,idx)=>{ P.push(Pt[0],Pt[1],Pt[2]); CR.push(cross); LN.push(s[idx]);
      COL.push(col[idx].r,col[idx].g,col[idx].b); WV.push(wd[0],wd[1],wd[2]); WP.push(wp[0],wp[1],wp[2],wp[3]); };
    for(let i=0;i<M-1;i++){
      pv(L[i],-1,i); pv(C[i],0,i); pv(C[i+1],0,i+1);
      pv(L[i],-1,i); pv(C[i+1],0,i+1);pv(L[i+1],-1,i+1);
      pv(C[i],0,i); pv(R[i],1,i); pv(R[i+1],1,i+1);
      pv(C[i],0,i); pv(R[i+1],1,i+1);pv(C[i+1],0,i+1);
    }
  }
  plane(N1); plane(N2);
}

// ----- flowing-hair math: fan out + sweep back + droop + waves -----
const rand=(a,b)=>a+Math.random()*(b-a);
function strand(theta, F, S, D, rise, cRoot, cMid, cTip, width, M, swayAmp, swayFreq){
  const c=Math.cos(theta), sn=Math.sin(theta);
  const r=[c,sn,0], tang=[-sn,c,0];
  const phase=Math.random()*6.28;
  const pts=[]; const steps=13;
  for(let k=0;k<steps;k++){
    const t=k/steps;
    const fan = F*Math.sqrt(t);
    const sweep = S*t;
    const droop = D*t*t;
    const up = rise*Math.sin(t*Math.PI*0.7);
    const env2 = 0.3 + t;
    const swL = (Math.sin(t*Math.PI*swayFreq+phase) + 0.4*Math.sin(t*Math.PI*swayFreq*2.3+phase*1.7))*swayAmp*env2;

```

```

    const swD = Math.cos(t*Math.PI*swayFreq*0.9+phase*1.3)*swayAmp*0.6*env2;
    pts.push([ r[0]*fan + tang[0]*swL,
               r[1]*fan + up - droop + tang[1]*swL,
               -sweep + swD ]]);
}
addStrand(pts, {M,width,taper:0.9,cRoot,cMid,cTip,speed:rand(0.4,0.9),waves:rand(1.2,2.4),amp:rand(0.05,0.14),phase});
}

function build(){
  // short fine fuzz right at the head (electric on top) – short, so it reads as a mane, not legs
  const MANE=58;
  for(let i=0;i<MANE;i++){
    const theta=Math.random()*Math.PI*2;
    const up=Math.sin(theta);
    const electric = up>0.35;
    const F=rand(0.35,0.8), S=rand(0.4,1.0), D=rand(0.2,0.6)*(up<-0.3?1.6:1);
    if(electric)
      strand(theta, F, S*0.8, D, rand(0.2,0.6), BLUEL, BLUE, MINT, rand(0.045,0.075), 16, rand(0.08,0.16), rand(2.0,3.2));
    else
      strand(theta, F, S, D, 0, CORE, MINT, TEAL, rand(0.05,0.085), 16, rand(0.08,0.16), rand(2.0,3.2));
  }
  // thick flowing hair locks – the body and tail
  const HAIR=58;
  for(let i=0;i<HAIR;i++){
    const theta=Math.random()*Math.PI*2;
    const up=Math.sin(theta);
    const sc=Math.pow(Math.random(),0.5);
    let F=(0.6+sc*1.2)*rand(0.85,1.15);
    let S=(1.2+sc*2.3)*rand(0.85,1.15);
    let D=(0.4+sc*1.0)*rand(0.85,1.15);
    if(up<-0.3) D*=1.5;
    strand(theta, F, S, D, 0, CORE, MINT, DEEP, rand(0.10,0.19), 24, rand(0.14,0.26), rand(1.6,2.8));
  }
  // bold hero locks – widest, longest, big graceful S-curves
  const BOLD=13;
  for(let i=0;i<BOLD;i++){
    const theta=(i/BOLD)*Math.PI*2 + rand(-0.25,0.25);
    const up=Math.sin(theta);
    let D=(0.6+Math.random()*1.0); if(up<-0.3) D*=1.4;
    strand(theta, rand(1.0,1.8), rand(2.4,3.6), D, 0, CORE, MINT, DEEP,
           rand(0.17,0.26), 28, rand(0.22,0.36), rand(1.5,2.6));
  }
}
build();

// ----- one merged mesh (soft painted hair, normal blending) -----
const geo = new THREE.BufferGeometry();
geo.setAttribute('position', new THREE.BufferAttribute(Float32Array.from(P),3));
geo.setAttribute('aCross', new THREE.BufferAttribute(Float32Array.from(CR),1));
geo.setAttribute('aLen', new THREE.BufferAttribute(Float32Array.from(LN),1));
geo.setAttribute('aColor', new THREE.BufferAttribute(Float32Array.from(COL),3));
geo.setAttribute('aWave', new THREE.BufferAttribute(Float32Array.from(WV),3));
geo.setAttribute('awparams', new THREE.BufferAttribute(Float32Array.from(WP),4));
geo.computeBoundingSphere();

const uniforms = { uTime:{value:0}, uSoft:{value:2.0}, uIntensity:{value:0.8} };
const mat = new THREE.ShaderMaterial({
  uniforms,
  vertexShader:`
    attribute float aCross; attribute float aLen; attribute vec3 aColor;
    attribute vec3 aWave; attribute vec4 awparams;
    uniform float uTime;
    varying float vC; varying float vL; varying vec3 vCol;
    void main(){

```



```

        float wv = sin(uTime*awparams.x + aLen*awparams.y + awparams.w) * awparams.z * aLen;
        vec3 p = position + awave * wv;
        vC=aCross; vL=aLen; vCol=aColor;
        gl_Position = projectionMatrix * modelViewMatrix * vec4(p,1.0);
    }`,
    fragmentShader:`
        uniform float uSoft; uniform float uIntensity;
        varying float vC; varying float vL; varying vec3 vCol;
        void main(){
            float edge = pow(max(0.0,1.0-abs(vC)), uSoft);
            float env = smoothstep(0.0,0.02,vL) * pow(1.0-vL,0.9);
            gl_FragColor = vec4(vCol, edge*env*uIntensity);
        }`,
    transparent:true, blending:THREE.NormalBlending,
    depthWrite:false, depthTest:true, side:THREE.DoubleSide
});
const mesh = new THREE.Mesh(geo, mat); mesh.frustumCulled=false; group.add(mesh);

// the one bright point from the drawing: a small pure-white core spot at the center
function coreTex(){
    const S=256, c=document.createElement('canvas'); c.width=c.height=S;
    const x=c.getContext('2d'), C=S/2;

    // --- STAR FLARE: sharp light spikes radiating from the core ---
    x.save(); x.translate(C,C);
    const spike=(ang,len,wid,alpha)>0;{
        x.save(); x.rotate(ang);
        const g=x.createLinearGradient(0,0,len,0);
        g.addColorStop(0, 'rgba(255,255,255,'+alpha+')');
        g.addColorStop(0.22, 'rgba(240,255,252,'+(alpha*0.70)+')');
        g.addColorStop(0.60, 'rgba(210,250,244,'+(alpha*0.26)+')');
        g.addColorStop(1, 'rgba(190,248,240,0)');
        x.fillStyle=g;
        x.beginPath(); x.moveTo(0,-wid); x.lineTo(len,0); x.lineTo(0,wid); x.closePath(); x.fill();
        x.restore();
    };
    for(let i=0;i<4;i++) spike(i*Math.PI/2, C*1.00, 8.5, 1.00); // 4 long main spikes
    for(let i=0;i<4;i++) spike(Math.PI/4+i*Math.PI/2, C*0.62, 5.0, 0.62); // 4 short diagonals
    x.restore();

    // --- CORE: small hard-edged bright centre ---
    const g=x.createRadialGradient(C,C,0,C,C,C);
    g.addColorStop(0, 'rgba(255,255,255,1)');
    g.addColorStop(0.20, 'rgba(255,255,255,0.98)');
    g.addColorStop(0.27, 'rgba(235,255,252,0.55)');
    g.addColorStop(0.36, 'rgba(200,250,242,0.10)');
    g.addColorStop(1, 'rgba(180,248,238,0)');
    x.globalCompositeOperation='lighter';
    x.fillStyle=g; x.fillRect(0,0,S,S);
    return new THREE.CanvasTexture(c);
}
const coreSprite = new THREE.Sprite(new THREE.SpriteMaterial({
    map:coreTex(), color:0xffffffff, transparent:true, opacity:0.9,
    blending:THREE.AdditiveBlending, depthTest:false, depthWrite:false }));
coreSprite.position.set(0, 0, 0.22); // sits partially in front of the star head
coreSprite.scale.setScalar(0.62);
group.add(coreSprite);

// ----- sparks: tiny sharp points that spawn ALONG the tendrils -----
const SPK=220, spos=[], sseed=[], scol=[], ssize=[];
const warm=[new THREE.Color(0xffe8b4),new THREE.Color(0xffcf82),new THREE.Color(0xffffffff),new THREE.Color(0xc4f4e8)];
for(let i=0;i<SPK;i++){
    const anch = SPARK_ANCHORS.length ? SPARK_ANCHORS[(Math.random()*SPARK_ANCHORS.length)|0] : [0,0,0];
    const j=0.06;

```

```

    spos.push(anch[0]+(Math.random()-0.5)*j, anch[1]+(Math.random()-0.5)*j, anch[2]+(Math.random()-0.5)*j);
    sseed.push(Math.random());
    const c=warm[Math.random()&lt;0.68?(Math.random()&lt;0.5?0:1):(Math.random()&lt;0.5?2:3)];
    scol.push(c.r,c.g,c.b);
    ssize.push(1.4+Math.random()*2.0);
}
const sgeo=new THREE.BufferGeometry();
sgeo.setAttribute('position',new THREE.BufferAttribute(Float32Array.from(spos),3));
sgeo.setAttribute('aSeed', new THREE.BufferAttribute(Float32Array.from(sseed),1));
sgeo.setAttribute('aColor',new THREE.BufferAttribute(Float32Array.from(scol),3));
sgeo.setAttribute('aSize', new THREE.BufferAttribute(Float32Array.from(ssize),1));
sgeo.computeBoundingSphere();
const smat=new THREE.ShaderMaterial({
    uniforms:{uTime:{value:0}, uDpr:{value:Math.min(devicePixelRatio,2)}},
    vertexShader:`
        attribute float aSeed; attribute vec3 aColor; attribute float aSize;
        uniform float uTime; uniform float uDpr; varying vec3 vCol; varying float vTw;
        void main(){
            float t=uTime*0.8 + aSeed*6.2831;
            vec3 p=position;
            p.x+=sin(t)*0.02; p.y+=cos(t*0.8)*0.02; p.z+=sin(t*1.1)*0.02;
            vCol=aColor; vTw=0.5+0.5*sin(t*3.2+aSeed*18.0);
            vec4 mv=modelViewMatrix*vec4(p,1.0);
            float ps=aSize*(0.8+0.6*vTw)*(8.0/-mv.z)*uDpr;
            gl_PointSize=clamp(ps,1.5,7.0*uDpr);
            gl_Position=projectionMatrix*mv;
        }`,
    fragmentShader:`
        varying vec3 vCol; varying float vTw;
        void main(){
            float d=length(gl_PointCoord-vec2(0.5)); if(d>0.5) discard;
            float core=smoothstep(0.5,0.0,d);
            gl_FragColor=vec4(vCol, pow(core,2.2)*(0.8+0.5*vTw));
        }`,
    transparent:true, blending:THREE.AdditiveBlending, depthTest:false, depthWrite:false
});
const sparks=new THREE.Points(sgeo,smat); sparks.frustumCulled=false; group.add(sparks);

// lights (only affect the solid head material; the shader-based body ignores them)
scene.add(new THREE.AmbientLight(0xffffffff, 0.55));
const _dl=new THREE.DirectionalLight(0xffffffff, 0.95); _dl.position.set(1.2,1.5,2.0); scene.add(_dl);

// ----- star HEAD: a real extruded solid (beveled = soft edges), curved back to follow the body (DEBUG red) -----
// ----- HEAD as MATH: smooth polar star field, soft glow (no mesh object, no lighting) -----
//  $R(a) = R_i + (R_o - R_i) * (0.5 + 0.5 * \cos(P * a))^k$  &lt;- infinitely smooth star, no straight edges
// thickness(u) =  $T * (1 - u)^q$  &lt;- fat centre, thin tips
//  $z = \text{curve} * (x^2 + y^2)$  &lt;- curls back around the body
function makeStarHead(size, curve, color){
    const NA=200, NR=40, Ro=1.0, Ri=0.46, P=4.0, K=3.2, T=0.20, Q=0.85;
    const Rf=a>0.5? Ri + (Ro-Ri)*Math.pow(0.5+0.5*Math.cos(P*a), K);
    const pos=[], uu=[];
    // ONE continuous analytic profile - no pieces, no join, no crease:
    //  $th(u) = T * \exp(-(u/W)^N) * (1 - u^{12})$ 
    // exp with N>2 is naturally FLAT through the middle (the face for the eye)
    // and rolls off organically; the last term takes it exactly to zero at the rim.
    const W=0.78, N=5.0;
    const th=u>0.5? T*Math.exp(-Math.pow(u/W,N))*(1-Math.pow(u,12));
    const V=(u,a,sgn)=>{
        const rad=u*Rf(a), x=Math.cos(a)*rad, y=Math.sin(a)*rad;
        const z=sgn*th(u) - curve*(x*x+y*y);
        pos.push(x,y,z); uu.push(u);
    };
    for(let sgn of [1,-1])
        for(let j=0;j<NR;j++){

```

```

const u0=j/NR, u1=(j+1)/NR;
for(let i=0;i<NA;i++){
  const a0=6.28318530718*i/NA, a1=6.28318530718*(i+1)/NA;
  V(u0,a0,sgn); V(u1,a0,sgn); V(u1,a1,sgn);
  V(u0,a0,sgn); V(u1,a1,sgn); V(u0,a1,sgn);
}
}
const g=new THREE.BufferGeometry();
g.setAttribute('position',new THREE.BufferAttribute(Float32Array.from(pos),3));
g.setAttribute('aU',new THREE.BufferAttribute(Float32Array.from(uu),1));
g.computeVertexNormals();
const mat=new THREE.ShaderMaterial({
  uniforms:{uColor:{value:new THREE.Color(color)}},
  vertexShader:`attribute float aU; varying float vU; varying vec3 vN; varying vec3 vV;
  void main(){
    vU=aU;
    vN = normalize(normalMatrix * normal);
    vec4 mv = modelViewMatrix * vec4(position,1.0);
    vV = normalize(-mv.xyz);
    gl_Position = projectionMatrix * mv;
  }`,
  fragmentShader:`uniform vec3 uColor; varying float vU; varying vec3 vN; varying vec3 vV;
  void main(){
    // EMISSIVE GLOW in the shader itself - no post-process, no sprite.
    float fres = pow(1.0 - abs(dot(normalize(vN), normalize(vV))), 2.2); // rim lights up
    vec3 col = uColor * (0.85 + 1.60*fres); // luminous toward the silhouette
    col = mix(col, vec3(1.0), fres*0.45); // hottest edges bleach toward white
    gl_FragColor = vec4(col, 1.0);
  }`,
  transparent:false, blending:THREE.NormalBlending,
  depthTest:true, depthWrite:true,
  side:THREE.DoubleSide });
const m=new THREE.Mesh(g,mat); m.frustumCulled=false; m.scale.setScalar(size);
return m;
}
const head = makeStarHead(0.56, 0.40, 0x7fec4); // tendril mint, solid // size, curve-back, DEBUG colour
head.position.set(0, 0, -0.02); // slight intersect with the body
group.add(head);

// ----- orbit camera -----
const target = new THREE.Vector3(0,-0.4,-0.9);
let theta=0, phi=Math.PI/2, radius=8;
function updateCam(){ const sp=Math.sin(phi);
  camera.position.set(target.x+radius*sp*Math.sin(theta), target.y+radius*Math.cos(phi), target.z+radius*sp*Math.cos(theta));
  camera.lookAt(target); }
function preset(v){ if(v==='front'){ theta=0; phi=Math.PI/2; radius=8; }
  else { theta=Math.PI/2; phi=Math.PI/2*0.94; radius=9.5; } updateCam(); }
updateCam();

let dragging=false, lx=0, ly=0;
const hint=document.getElementById('hint'); let hHid=false;
function hideHint(){ if(!hHid){ hHid=true; hint.classList.add('fade'); } }
renderer.domElement.addEventListener('pointerdown', e=>{ dragging=true; lx=e.clientX; ly=e.clientY; document.body.classList.add('drag'); hideHint(); });
addEventListener('pointerup', ()=>{ dragging=false; document.body.classList.remove('drag'); });
addEventListener('pointermove', e=>{ if(!dragging) return;
  const dx=e.clientX-lx, dy=e.clientY-ly; lx=e.clientX; ly=e.clientY;
  theta -= dx*0.006; phi = Math.min(Math.PI-0.12, Math.max(0.12, phi - dy*0.006)); updateCam(); });
addEventListener('wheel', e=>{ radius=THREE.MathUtils.clamp(radius+e.deltaY*0.003,4,16); updateCam(); }, {passive:true});
addEventListener('resize', ()=>{ camera.aspect=innerWidth/innerHeight; camera.updateProjectionMatrix();
  renderer.setSize(innerWidth,innerHeight); renderer.setPixelRatio(Math.min(devicePixelRatio,2)); });

document.getElementById('bFront').onclick={()=>{ setBtn('front'); preset('front'); }};
document.getElementById('bSide').onclick={()=>{ setBtn('side'); preset('side'); }};
function setBtn(v){ document.getElementById('bFront').classList.toggle('on',v==='front');

```

```

    document.getElementById('bSide').classList.toggle('on',v==='side'); }
    setTimeout(hideHint, 5200);

// ----- EYE MOVEMENT: saccade. Pick a spot, snap to it, hold dead still an
//          uneven beat, flick elsewhere. Sometimes startle and glance wide. -----
const EYE_BASE={x:0, y:0, z:0.06};
const EYE_LOOK=0.13;           // how far it can roam inside the head
const EYE_DART_SPEED=5.5;       // glide speed to a new spot (calm, not startled)
const EYE_HOLD_MIN=1.20;       // shortest pause, seconds
const EYE_HOLD_MAX=5.5;        // longest pause, seconds
const EYE_STARTLE_CHANCE=0.06; // odds any given glance is a big one
let eyeLookX=0, eyeLookY=0, eyeTargetX=0, eyeTargetY=0, eyeNextDart=0;

const clock=new THREE.Clock();
(function loop(){
    const delta=clock.getDelta();
    const t=clock.getElapsedTime(); uniforms.uTime.value=t; smat.uniforms.uTime.value=t;
    coreSprite.scale.setScalar(0.52 + 0.06*Math.sin(t*1.6));
    coreSprite.material.opacity = 0.82 + 0.14*Math.sin(t*1.6);

    if(t>eyeNextDart){
        const reach = Math.random()<EYE_STARTLE_CHANCE ? 1.0 : 0.55;
        const angle = Math.random()*Math.PI*2;
        const dist = Math.sqrt(Math.random())*reach*EYE_LOOK; // sqrt spreads glances evenly
        eyeTargetX = Math.cos(angle)*dist;
        eyeTargetY = Math.sin(angle)*dist;
        eyeNextDart = t + EYE_HOLD_MIN + Math.random()*(EYE_HOLD_MAX-EYE_HOLD_MIN);
    }
    const dartT = 1 - Math.exp(-EYE_DART_SPEED*delta);
    eyeLookX += (eyeTargetX-eyeLookX)*dartT;
    eyeLookY += (eyeTargetY-eyeLookY)*dartT;
    coreSprite.position.set(EYE_BASE.x+eyeLookX, EYE_BASE.y+eyeLookY, EYE_BASE.z);

    renderer.render(scene,camera);
    requestAnimationFrame(loop);
})();
})();
</script>
</body>
</html>

```